Capítulo 02 - Arquitetura de Software Detalhada

2.1. Introdução à Arquitetura do Sistema

A arquitetura do Projeto Quimera foi cuidadosamente projetada para atender a requisitos complexos e, muitas vezes, conflitantes. Por um lado, o sistema precisa ser **altamente escalável** para processar centenas de perfis por dia. Por outro, precisa ser **furtivo e discreto**, operando de forma que simule perfeitamente o comportamento humano para evitar detecção. Além disso, deve ser **resiliente** a falhas, **manutenível** por desenvolvedores e **observável** para debugging e otimização.

Para atender a esses requisitos, adotamos uma **arquitetura de microserviços baseada em agentes**, combinada com padrões arquiteturais modernos como Event-Driven Architecture (EDA), CQRS (Command Query Responsibility Segregation) e Domain-Driven Design (DDD). Esta combinação resulta em um sistema modular, desacoplado e altamente coeso, onde cada componente tem uma responsabilidade única e bem definida.

2.1.1. Princípios Arquiteturais Fundamentais

A arquitetura do Projeto Quimera é guiada por seis princípios fundamentais, derivados das melhores práticas de engenharia de software e arquitetura de sistemas distribuídos:

1. Single Responsibility Principle (SRP)

Cada agente e serviço tem uma única responsabilidade claramente definida. O Agente Batedor, por exemplo, é responsável **apenas** pela coleta de URLs de perfis. Ele não analisa perfis, não gera mensagens, não envia mensagens. Essa separação de responsabilidades torna o código mais coeso, testável e manutenível.

Benefício: Quando há um bug na coleta de URLs, sabemos exatamente onde procurar (Agente Batedor). Quando precisamos otimizar a análise de perfis, modificamos apenas o Agente Analista, sem risco de afetar outros componentes.

2. Loose Coupling (Baixo Acoplamento)

Os agentes e serviços são fracamente acoplados, comunicando-se principalmente via eventos assíncronos (message queues) e, quando necessário, via APIs REST. Um agente não conhece a implementação interna de outro agente; ele apenas emite eventos e consome eventos.

Exemplo: O Agente Batedor, ao terminar de coletar URLs, emite um evento `PerfilColetado` para uma fila Redis. Ele não sabe que o Agente Analista irá consumir esse evento. Se

amanhã decidirmos adicionar um novo agente que também processa perfis coletados, basta fazê-lo subscrever à mesma fila. O Agente Batedor não precisa ser modificado.

Benefício: Facilita a evolução do sistema. Podemos adicionar, remover ou modificar agentes sem causar efeito cascata em outros componentes.

3. High Cohesion (Alta Coesão)

Dentro de cada agente, as funções e classes estão fortemente relacionadas e trabalham juntas para cumprir a responsabilidade do agente. Não há código "estranho" ou funcionalidades não relacionadas dentro de um agente.

Exemplo: O Agente Analista contém classes para extração de features de texto (NLP), extração de features visuais (Computer Vision), cálculo de pontuação e identificação de afinidades. Todas essas classes estão relacionadas à responsabilidade central do agente: analisar perfis.

Benefício: Código mais legível, testável e reutilizável. Desenvolvedores conseguem entender rapidamente o que um agente faz sem precisar navegar por código não relacionado.

4. Fail-Fast e Resilience

O sistema é projetado para **falhar rapidamente** quando detecta um erro irrecuperável, mas também para ser **resiliente** a falhas transientes. Utilizamos padrões como Circuit Breaker, Retry com Exponential Backoff e Dead Letter Queues para lidar com falhas de forma elegante.

Exemplo: Se o Agente de Execução tenta enviar uma mensagem e recebe um erro HTTP 429 (Too Many Requests), ele não falha imediatamente. Em vez disso, ele aguarda um tempo exponencialmente crescente (1s, 2s, 4s, 8s...) e tenta novamente até 5 vezes. Se após 5 tentativas ainda falhar, a mensagem é movida para uma Dead Letter Queue para análise posterior.

Benefício: O sistema continua operando mesmo quando componentes individuais falham temporariamente. Falhas transientes (rede instável, API temporariamente indisponível) não derrubam o sistema.

5. Observability (Observabilidade)

Cada componente do sistema emite **métricas**, **logs estruturados** e **traces distribuídos**. Isso permite que desenvolvedores e operadores entendam o que está acontecendo dentro do sistema em tempo real, identifiquem gargalos e debuguem problemas rapidamente.

Exemplo: Quando o Agente Analista processa um perfil, ele emite:

- **Métrica:** `perfis_analisados_total` (contador)
- **Métrica:** `tempo_analise_segundos` (histograma)

- **Log:** `{"agente": "analista", "acao": "analisar_perfil", "perfil_id": "abc123", "pontuacao": 8.5, "tempo_ms": 1234}`
- **Trace:** Span com ID único que rastreia a requisição desde a coleta até o envio da mensagem

Benefício: Visibilidade completa do sistema. Podemos responder perguntas como "Por que a taxa de resposta caiu 10% esta semana?" ou "Qual agente está causando o gargalo?" com dados concretos.

6. Security by Design (Segurança por Design)

Segurança não é uma camada adicional aplicada no final; ela é incorporada desde o início do design. Dados sensíveis são criptografados em repouso e em trânsito. Acesso a APIs é protegido por autenticação e autorização. Logs não contêm informações sensíveis (senhas, tokens).

Exemplo: Senhas de banco de dados e API keys são armazenadas em variáveis de ambiente (ou HashiCorp Vault em produção), nunca hardcoded no código. Comunicação entre agentes e bancos de dados usa TLS. Logs substituem tokens por `[REDACTED]`.

Benefício: Reduz a superfície de ataque e protege dados sensíveis de vazamentos acidentais.

2.2. Padrões Arquiteturais Aplicados

2.2.1. Microservices Architecture (Arquitetura de Microserviços)

O Projeto Quimera adota uma arquitetura de microserviços, onde o sistema é decomposto em **serviços pequenos, independentes e fracamente acoplados**, cada um executando em seu próprio processo e comunicando-se via protocolos leves (HTTP, message queues).

Decomposição em Microserviços

O sistema é decomposto em **14 microserviços principais**:

Microserviço	Responsabilidade	Tecnologia	Co
API Backend	Interface REST/WebSocket para frontend	FastAPI	HT
Orquestrador	Coordenação de workflow e estado	Python	Re
Agente Batedor	Coleta de URLs de perfis	Python + Selenium	Re
Agente Analista	Análise e pontuação de perfis	Python + ML	Re
Motor de Persuasão	Definição de estratégia de comunicação	Python	Re
Agente Escriba	Geração de mensagens com LLM	Python + OpenAI	Re
Agente de Execução	Envio automatizado de mensagens	Python + Selenium	Re
Agente de Monitoramento	Detecção de respostas	Python + Selenium	Re
Agente de Aprendizagem	Retreinamento de modelos	Python + ML	Re
Serviço de Handoff	Preparação de pacote de contexto	Python	HT
Serviço de Notificações	Envio de alertas	Python	HT
Serviço de Anti-Detecção	Gerenciamento de proxies e fingerprints	Python	HT
PostgreSQL	Banco de dados relacional	PostgreSQL 15	TC
MongoDB	Banco de dados de documentos	MongoDB 7	TC
Redis	Cache e message broker	Redis 7	TC
ChromaDB	Banco vetorial	ChromaDB	HT

Benefícios da Arquitetura de Microserviços

1. Escalabilidade Independente

Cada microserviço pode ser escalado independentemente baseado em sua carga. Se o Agente Analista se tornar um gargalo (porque análise de perfis é computacionalmente intensiva), podemos escalar apenas esse agente, adicionando mais instâncias, sem precisar escalar os outros agentes.

Exemplo de Escalamento:

```yaml
```

```yaml
# docker-compose.yml
services:
  agente_analista:
    image: quimera/agente_analista:latest
    deploy:
      replicas: 5  # 5 instâncias do Agente Analista
    # ...

  agente_batedor:
    image: quimera/agente_batedor:latest
    deploy:
      replicas: 2  # Apenas 2 instâncias do Agente Batedor
    # ...
```

2. Resiliência a Falhas

Se um microserviço falha (crash, bug, out of memory), os outros continuam operando. O sistema degrada graciosamente em vez de falhar completamente.

Exemplo: Se o Agente Escriba crashar devido a um bug, os outros agentes (Batedor, Analista, Motor de Persuasão) continuam operando normalmente. Perfis continuam sendo coletados e analisados. Quando o Agente Escriba for reiniciado, ele processará as mensagens pendentes na fila.

3. Tecnologias Heterogêneas

Cada microserviço pode usar a tecnologia mais adequada para sua responsabilidade. Embora a maioria dos agentes seja implementada em Python (devido ao ecossistema de IA/ML), poderíamos implementar o Serviço de Anti-Detecção em Go (para melhor performance) ou o frontend em TypeScript/React.

4. Desenvolvimento e Deploy Independentes

Equipes diferentes podem trabalhar em microserviços diferentes sem conflitos. Cada microserviço pode ser desenvolvido, testado e deployado independentemente, acelerando o ciclo de desenvolvimento.

5. Manutenibilidade

Código modular e coeso é mais fácil de entender, testar e manter. Um desenvolvedor pode se tornar especialista em um microserviço específico sem precisar entender todo o sistema.

Desafios da Arquitetura de Microserviços

1. Complexidade Operacional

Gerenciar 14+ microserviços é mais complexo que gerenciar uma aplicação monolítica. Precisamos de ferramentas de orquestração (Docker Compose, Kubernetes),

monitoramento (Prometheus, Grafana), logging centralizado (ELK, Loki) e tracing distribuído (Jaeger).

Mitigação: Investir em automação (CI/CD), infraestrutura como código (Docker Compose, Terraform) e observabilidade (métricas, logs, traces).

2. Latência de Comunicação

Comunicação entre microserviços via rede adiciona latência comparado a chamadas de função em um monolito. Cada hop adiciona ~1-10ms de latência.

Mitigação: Usar comunicação assíncrona (message queues) quando possível, minimizar chamadas síncronas (HTTP) e otimizar serialização/deserialização de dados.

3. Consistência de Dados

Em um monolito, transações ACID garantem consistência. Em microserviços, cada serviço tem seu próprio banco de dados, e manter consistência entre eles é mais difícil.

Mitigação: Adotar **consistência eventual** (eventual consistency) e usar padrões como Saga Pattern para transações distribuídas.

4. Debugging Distribuído

Debugar um bug que envolve múltiplos microserviços é mais difícil que debugar um monolito. Um erro pode se propagar através de vários serviços antes de se manifestar.

Mitigação: Implementar tracing distribuído (Jaeger, Zipkin) para rastrear requisições através de múltiplos serviços. Usar correlation IDs para correlacionar logs de diferentes serviços.

2.2.2. Event-Driven Architecture (Arquitetura Orientada a Eventos)

O Projeto Quimera opera baseado em **eventos**. Quando algo significativo acontece no sistema (um perfil é coletado, uma análise é concluída, uma resposta é detectada), um **evento** é emitido. Outros componentes que estão interessados nesse evento podem subscrevê-lo e reagir a ele.

Fluxo de Eventos

Plain Text

```
┌──────────────────┐
│ Agente Batedor   │
│ coleta URL       │
└──────────────────┘
         │
         │
         │ emite evento
         ▼
```

```
    PerfilColetado
          │
          │ consome evento
          ▼
┌──────────────────┐
│ Agente Analista  │
│ analisa perfil   │
└──────────────────┘
          │
          │
          │ emite evento
          ▼
   PerfilAnalisado
          │
          │ consome evento
          ▼
┌──────────────────┐
│ Motor Persuasão  │
│ define estratégia│
└──────────────────┘
          │
          │
          │ emite evento
          ▼
 EstratégiaDefinida
          │
          │ consome evento
          ▼
┌──────────────────┐
│ Agente Escriba   │
│ gera mensagem    │
└──────────────────┘
          │
          │
          │ emite evento
          ▼
  MensagemGerada
          │
          │ consome evento
          ▼
┌──────────────────┐
│ Agente Execução  │
│ envia mensagem   │
└──────────────────┘
          │
          │
          │ emite evento
          ▼
  MensagemEnviada
          │
          │ consome evento
          ▼
```

```
┌──────────────────┐
│ Agente Monitor   │
│ monitora inbox   │
└──────────────────┘
         │
         │
         │ emite evento (se resposta)
         ▼
 RespostaDetectada
         │
         │ consome evento
         ▼
┌──────────────────┐
│ Serviço Handoff  │
│ prepara contexto │
└──────────────────┘
```

Implementação com Redis Pub/Sub

Utilizamos **Redis Pub/Sub** como message broker para comunicação assíncrona entre microserviços. Redis foi escolhido por sua simplicidade, baixa latência (~1ms) e capacidade de lidar com alta throughput (100K+ mensagens/segundo).

Estrutura de Eventos:

Cada evento é um objeto JSON com a seguinte estrutura:

JSON

```json
{
  "event_type": "PerfilAnalisado",
  "event_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2025-10-10T16:30:00Z",
  "source": "agente_analista",
  "data": {
    "perfil_id": "abc123",
    "pontuacao_final": 8.5,
    "status": "qualificado"
  },
  "metadata": {
    "correlation_id": "req-789xyz",
    "user_id": "user-456"
  }
}
```

Publicação de Eventos (Publisher):

Python

```python
# agente_analista/publisher.py
import redis
import json
from datetime import datetime
from uuid import uuid4

class EventPublisher:
    def __init__(self, redis_url: str):
        self.redis = redis.from_url(redis_url)

    def publicar_evento(
        self,
        event_type: str,
        data: dict,
        source: str,
        correlation_id: str = None
    ):
        """Publica um evento no canal Redis."""
        evento = {
            "event_type": event_type,
            "event_id": str(uuid4()),
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "source": source,
            "data": data,
            "metadata": {
                "correlation_id": correlation_id or str(uuid4())
            }
        }

        # Publicar no canal específico do tipo de evento
        canal = f"eventos:{event_type}"
        self.redis.publish(canal, json.dumps(evento))

        # Também publicar no canal geral para auditoria
        self.redis.publish("eventos:todos", json.dumps(evento))

        print(f"[EVENTO PUBLICADO] {event_type} - {evento['event_id']}")

# Uso no Agente Analista
publisher = EventPublisher(redis_url="redis://:senha@redis:6379/0")

# Após analisar um perfil
publisher.publicar_evento(
    event_type="PerfilAnalisado",
    data={
        "perfil_id": perfil.id,
        "pontuacao_final": perfil.pontuacao_final,
```

```
        "status": perfil.status
    },
    source="agente_analista",
    correlation_id=correlation_id
)
```

Consumo de Eventos (Subscriber):

Python

```python
# motor_persuasao/subscriber.py
import redis
import json
import threading

class EventSubscriber:
    def __init__(self, redis_url: str):
        self.redis = redis.from_url(redis_url)
        self.pubsub = self.redis.pubsub()
        self.handlers = {}

    def registrar_handler(self, event_type: str, handler_func):
        """Registra uma função handler para um tipo de evento."""
        self.handlers[event_type] = handler_func

    def subscrever(self, event_types: list):
        """Subscreve a um ou mais tipos de eventos."""
        canais = [f"eventos:{et}" for et in event_types]
        self.pubsub.subscribe(*canais)
        print(f"[SUBSCRITO] {', '.join(canais)}")

    def iniciar(self):
        """Inicia o loop de consumo de eventos."""
        print("[SUBSCRIBER] Aguardando eventos...")

        for mensagem in self.pubsub.listen():
            if mensagem['type'] == 'message':
                self._processar_mensagem(mensagem)

    def _processar_mensagem(self, mensagem):
        """Processa uma mensagem recebida."""
        try:
            evento = json.loads(mensagem['data'])
            event_type = evento['event_type']

            if event_type in self.handlers:
                print(f"[EVENTO RECEBIDO] {event_type} -
{evento['event_id']}")
```

```
                self.handlers[event_type](evento)
            else:
                print(f"[AVISO] Nenhum handler para evento: {event_type}")

        except Exception as e:
            print(f"[ERRO] Falha ao processar mensagem: {e}")

# Uso no Motor de Persuasão
subscriber = EventSubscriber(redis_url="redis://:senha@redis:6379/0")

def processar_perfil_analisado(evento):
    """Handler para evento PerfilAnalisado."""
    perfil_id = evento['data']['perfil_id']
    pontuacao = evento['data']['pontuacao_final']

    if pontuacao >= 7.5:  # Apenas perfis qualificados
        print(f"[MOTOR] Definindo estratégia para perfil {perfil_id}")
        # Lógica de definição de estratégia...
        estrategia = definir_estrategia(perfil_id)

        # Publicar evento de estratégia definida
        publisher.publicar_evento(
            event_type="EstratégiaDefinida",
            data={
                "perfil_id": perfil_id,
                "estrategia_id": estrategia.id
            },
            source="motor_persuasao",
            correlation_id=evento['metadata']['correlation_id']
        )

# Registrar handler e subscrever
subscriber.registrar_handler("PerfilAnalisado", processar_perfil_analisado)
subscriber.subscrever(["PerfilAnalisado"])

# Iniciar em thread separada
thread = threading.Thread(target=subscriber.iniciar, daemon=True)
thread.start()
```

Benefícios da Arquitetura Orientada a Eventos

1. Desacoplamento Temporal

O publisher não precisa esperar que o subscriber processe o evento. Ele publica o evento e continua sua execução. Isso melhora a responsividade e throughput do sistema.

2. Desacoplamento de Componentes

O publisher não conhece os subscribers, e vice-versa. Podemos adicionar novos subscribers sem modificar o publisher.

Exemplo: Se quisermos adicionar um novo agente que também reage a `PerfilAnalisado` (por exemplo, um agente de auditoria que registra todas as análises), basta fazê-lo subscrever ao evento. Nenhum código existente precisa ser modificado.

3. Escalabilidade

Múltiplos subscribers podem consumir o mesmo evento em paralelo, distribuindo a carga de processamento.

4. Auditoria e Replay

Todos os eventos são publicados no canal `eventos:todos`, criando um log de auditoria completo. Podemos usar esse log para replay (reprocessar eventos) em caso de falha ou para análise.

Desafios da Arquitetura Orientada a Eventos

1. Complexidade de Debugging

Fluxos assíncronos são mais difíceis de debugar que fluxos síncronos. Um evento pode ser processado por múltiplos subscribers, e rastrear o fluxo completo requer correlation IDs e tracing distribuído.

2. Garantias de Entrega

Redis Pub/Sub **não garante entrega** de mensagens. Se um subscriber estiver offline quando um evento é publicado, ele perderá o evento. Para garantias mais fortes, precisaríamos usar um message broker mais robusto (RabbitMQ, Kafka).

Mitigação: Para eventos críticos, usar **Redis Streams** (que persiste mensagens) ou **RabbitMQ** em vez de Pub/Sub.

3. Ordenação de Eventos

Redis Pub/Sub não garante ordenação de eventos entre diferentes canais. Se a ordem importa, precisamos usar um único canal ou adicionar lógica de ordenação no subscriber.

2.2.3. CQRS (Command Query Responsibility Segregation)

CQRS é um padrão que separa operações de **escrita (comandos)** de operações de **leitura (queries)**, permitindo otimizar cada uma independentemente.

Separação de Comandos e Queries

Comandos (Write Model):

Comandos são operações que **modificam o estado** do sistema. Eles são processados de forma síncrona e transacional.

Exemplos de Comandos:

- `CriarPerfil`
- `AtualizarPontuacao`
- `EnviarMensagem`
- `RegistrarFeedback`

Queries (Read Model):

Queries são operações que **leem o estado** do sistema sem modificá-lo. Elas são otimizadas para leitura rápida, muitas vezes usando views materializadas ou cache.

Exemplos de Queries:

- `ListarPerfisQualificados`
- `BuscarConversasAtivas`
- `ObterMetricasDashboard`
- `BuscarPerfilPorId`

Implementação de CQRS

Camada de Comando (Write):

Python

```python
# comandos/criar_perfil.py
from dataclasses import dataclass
from uuid import UUID, uuid4
from datetime import datetime

@dataclass
class CriarPerfilComando:
    """Comando para criar um novo perfil."""
    url: str
    plataforma: str
    hotspot_origem_id: UUID
    usuario_id: UUID

class CriarPerfilHandler:
    """Handler que processa o comando CriarPerfil."""

    def __init__(self, perfil_repository, event_publisher):
        self.perfil_repository = perfil_repository
```

```python
        self.event_publisher = event_publisher

    def executar(self, comando: CriarPerfilComando) -> UUID:
        """Executa o comando e retorna o ID do perfil criado."""

        # Validar se perfil já existe
        perfil_existente = self.perfil_repository.buscar_por_url(comando.url)
        if perfil_existente:
            raise ValueError(f"Perfil já existe: {comando.url}")

        # Criar perfil
        perfil = Perfil(
            id=uuid4(),
            url=comando.url,
            plataforma=comando.plataforma,
            hotspot_origem_id=comando.hotspot_origem_id,
            usuario_id=comando.usuario_id,
            status_processamento="descoberto",
            timestamp_coleta=datetime.utcnow()
        )

        # Persistir no banco de dados (write model)
        self.perfil_repository.criar(perfil)

        # Publicar evento
        self.event_publisher.publicar_evento(
            event_type="PerfilColetado",
            data={
                "perfil_id": str(perfil.id),
                "url": perfil.url,
                "plataforma": perfil.plataforma
            },
            source="comando_criar_perfil"
        )

        return perfil.id
```

Camada de Query (Read):

Python

```python
# queries/listar_perfis_qualificados.py
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class ListarPerfisQualificadosQuery:
    """Query para listar perfis qualificados."""
```

```python
    usuario_id: str
    limite: int = 50
    offset: int = 0
    pontuacao_minima: Optional[float] = None

@dataclass
class PerfilQualificadoDTO:
    """Data Transfer Object para perfil qualificado."""
    id: str
    nome_completo: str
    foto_perfil_url: str
    pontuacao_final: float
    pontuacao_interessante: float
    pontuacao_bonita: float
    pontuacao_solteira: float
    timestamp_analise: str

class ListarPerfisQualificadosHandler:
    """Handler que processa a query ListarPerfisQualificados."""

    def __init__(self, read_db_connection):
        self.db = read_db_connection

    def executar(self, query: ListarPerfisQualificadosQuery) ->
List[PerfilQualificadoDTO]:
        """Executa a query e retorna lista de perfis."""

        # Query otimizada no read model (view materializada)
        sql = """
            SELECT
                p.id,
                p.nome_completo,
                p.foto_perfil_url,
                p.pontuacao_final,
                p.pontuacao_interessante,
                p.pontuacao_bonita,
                p.pontuacao_solteira,
                p.timestamp_analise
            FROM vw_perfis_qualificados p
            WHERE p.usuario_id = %s
              AND p.status_processamento = 'qualificado'
              AND (%s IS NULL OR p.pontuacao_final >= %s)
            ORDER BY p.pontuacao_final DESC
            LIMIT %s OFFSET %s
        """

        resultados = self.db.execute(
            sql,
```

```
            (
                query.usuario_id,
                query.pontuacao_minima,
                query.pontuacao_minima,
                query.limite,
                query.offset
            )
        )

        return [
            PerfilQualificadoDTO(
                id=str(r['id']),
                nome_completo=r['nome_completo'],
                foto_perfil_url=r['foto_perfil_url'],
                pontuacao_final=float(r['pontuacao_final']),
                pontuacao_interessante=float(r['pontuacao_interessante']),
                pontuacao_bonita=float(r['pontuacao_bonita']),
                pontuacao_solteira=float(r['pontuacao_solteira']),
                timestamp_analise=r['timestamp_analise'].isoformat()
            )
            for r in resultados
        ]
```

View Materializada para Otimização de Leitura:

SQL

```sql
-- View materializada para perfis qualificados
CREATE MATERIALIZED VIEW vw_perfis_qualificados AS
SELECT
    p.id,
    p.usuario_id,
    p.nome_completo,
    p.foto_perfil_url,
    p.pontuacao_final,
    p.pontuacao_interessante,
    p.pontuacao_bonita,
    p.pontuacao_solteira,
    p.status_processamento,
    p.timestamp_analise,
    p.ranking,
    COUNT(pa.id) as total_afinidades
FROM perfis p
LEFT JOIN pontos_afinidade pa ON pa.perfil_id = p.id
WHERE p.status_processamento IN ('qualificado', 'contatado', 'respondeu')
GROUP BY p.id
ORDER BY p.pontuacao_final DESC;
```

```sql
-- Índice para acelerar queries
CREATE INDEX idx_vw_perfis_qualificados_usuario
ON vw_perfis_qualificados(usuario_id, pontuacao_final DESC);

-- Refresh automático a cada 5 minutos
CREATE OR REPLACE FUNCTION refresh_vw_perfis_qualificados()
RETURNS void AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY vw_perfis_qualificados;
END;
$$ LANGUAGE plpgsql;

-- Agendar refresh
SELECT cron.schedule('refresh-perfis-qualificados', '*/5 * * * *',
    'SELECT refresh_vw_perfis_qualificados()');
```

Benefícios de CQRS

1. Performance Otimizada

Queries podem ser otimizadas independentemente de comandos. Podemos usar views materializadas, cache agressivo, índices específicos e até bancos de dados diferentes (ex: PostgreSQL para write, Elasticsearch para read).

2. Escalabilidade

Read model e write model podem ser escalados independentemente. Como a maioria dos sistemas tem muito mais leituras que escritas, podemos ter múltiplas réplicas do read model.

3. Simplicidade de Queries

Queries podem ser simples e diretas, sem lógica de negócio complexa. Elas apenas leem dados pré-processados.

4. Flexibilidade

Podemos ter múltiplos read models otimizados para diferentes casos de uso. Por exemplo, um read model para o dashboard (agregações), outro para busca (Elasticsearch), outro para API (cache Redis).

Desafios de CQRS

1. Consistência Eventual

O read model pode estar temporariamente desatualizado em relação ao write model. Após um comando ser executado, pode levar alguns segundos até que a view materializada seja atualizada.

Mitigação: Aceitar consistência eventual para a maioria dos casos de uso. Para casos críticos, ler diretamente do write model.

2. Complexidade

CQRS adiciona complexidade ao sistema. Precisamos manter dois modelos (write e read) sincronizados.

Mitigação: Usar CQRS apenas onde os benefícios justificam a complexidade. Não aplicar CQRS em todo o sistema.

2.2.4. Domain-Driven Design (DDD)

Domain-Driven Design é uma abordagem de design de software que foca em modelar o **domínio do negócio** de forma rica e expressiva, usando uma linguagem ubíqua compartilhada entre desenvolvedores e especialistas do domínio.

Bounded Contexts (Contextos Delimitados)

O sistema é dividido em **bounded contexts**, cada um com seu próprio modelo de domínio, linguagem ubíqua e limites claros.

Bounded Contexts do Projeto Quimera:

1. **Contexto de Prospecção**
   - Entidades: Hotspot, Perfil (descoberto)
   - Linguagem: "coletar", "varrer", "extrair URLs"
   - Responsável: Agente Batedor
2. **Contexto de Análise**
   - Entidades: Perfil (analisado), Análise, PontoAfinidade
   - Linguagem: "analisar", "pontuar", "qualificar", "inferir"
   - Responsável: Agente Analista
3. **Contexto de Comunicação**
   - Entidades: Estratégia, Mensagem, Conversa
   - Linguagem: "persuadir", "gerar", "enviar", "monitorar"
   - Responsáveis: Motor de Persuasão, Agente Escriba, Agente de Execução, Agente de Monitoramento
4. **Contexto de Aprendizagem**
   - Entidades: Episódio, Feedback, Modelo
   - Linguagem: "treinar", "retreinar", "ajustar", "otimizar"

- Responsável: Agente de Aprendizagem

Entidades e Value Objects

Entidade: Objeto com identidade única que persiste ao longo do tempo.

Python

```python
# dominio/entidades/perfil.py
from dataclasses import dataclass
from uuid import UUID
from datetime import datetime
from typing import Optional

@dataclass
class Perfil:
    """Entidade Perfil - representa um perfil de rede social."""

    # Identidade
    id: UUID

    # Atributos básicos
    url: str
    plataforma: str  # 'instagram' ou 'facebook'
    username: str
    nome_completo: Optional[str]
    biografia: Optional[str]
    localizacao: Optional[str]
    foto_perfil_url: Optional[str]

    # Métricas sociais
    seguidores: int
    seguindo: int
    total_postagens: int

    # Pontuação
    pontuacao_interessante: Optional[float]
    pontuacao_bonita: Optional[float]
    pontuacao_solteira: Optional[float]
    pontuacao_final: Optional[float]
    confianca: Optional[float]
    ranking: Optional[int]

    # Status e timestamps
    status_processamento: str  # 'descoberto', 'em_analise', 'qualificado',
etc.
    timestamp_coleta: datetime
    timestamp_analise: Optional[datetime]
```

```
    timestamp_contato: Optional[datetime]
    timestamp_resposta: Optional[datetime]

    # Relacionamentos
    hotspot_origem_id: UUID
    usuario_id: UUID

    def qualificar(self, pontuacao_final: float, limiar: float):
        """Qualifica ou desqualifica o perfil baseado na pontuação."""
        if pontuacao_final >= limiar:
            self.status_processamento = "qualificado"
        else:
            self.status_processamento = "desqualificado"

        self.pontuacao_final = pontuacao_final
        self.timestamp_analise = datetime.utcnow()

    def marcar_como_contatado(self):
        """Marca o perfil como contatado."""
        if self.status_processamento != "qualificado":
            raise ValueError("Apenas perfis qualificados podem ser
contatados")

        self.status_processamento = "contatado"
        self.timestamp_contato = datetime.utcnow()

    def registrar_resposta(self):
        """Registra que o perfil respondeu."""
        if self.status_processamento != "contatado":
            raise ValueError("Apenas perfis contatados podem responder")

        self.status_processamento = "respondeu"
        self.timestamp_resposta = datetime.utcnow()
```

Value Object: Objeto sem identidade própria, definido apenas por seus atributos.

Python

```
# dominio/value_objects/pontuacao.py
from dataclasses import dataclass

@dataclass(frozen=True)  # Imutável
class Pontuacao:
    """Value Object que representa uma pontuação."""

    interessante: float  # 0-10
    bonita: float  # 0-10
    solteira: float  # 0-10
```

```python
    def __post_init__(self):
        """Valida os valores após inicialização."""
        for campo in ['interessante', 'bonita', 'solteira']:
            valor = getattr(self, campo)
            if not (0 <= valor <= 10):
                raise ValueError(f"{campo} deve estar entre 0 e 10,
recebido: {valor}")

    def calcular_final(self, pesos: 'PesosPontuacao') -> float:
        """Calcula a pontuação final baseada nos pesos."""
        return (
            self.interessante * pesos.peso_interessante +
            self.bonita * pesos.peso_bonita +
            self.solteira * pesos.peso_solteira
        )

@dataclass(frozen=True)
class PesosPontuacao:
    """Value Object que representa os pesos de cada critério."""

    peso_interessante: float  # 0-1
    peso_bonita: float  # 0-1
    peso_solteira: float  # 0-1

    def __post_init__(self):
        """Valida que a soma dos pesos é 1.0."""
        soma = self.peso_interessante + self.peso_bonita + self.peso_solteira
        if not (0.99 <= soma <= 1.01):  # Tolerância para erros de ponto
flutuante
            raise ValueError(f"Soma dos pesos deve ser 1.0, recebido:
{soma}")
```

Aggregates e Aggregate Roots

Aggregate: Cluster de entidades e value objects tratados como uma unidade para mudanças de dados.

Aggregate Root: Entidade que é o ponto de entrada para o aggregate.

Python

```python
# dominio/aggregates/conversa.py
from dataclasses import dataclass, field
from uuid import UUID, uuid4
from datetime import datetime
from typing import List, Optional
```

```python
@dataclass
class Mensagem:
    """Entidade Mensagem dentro do aggregate Conversa."""
    id: UUID
    remetente: str  # 'agente_escriba', 'usuario_humano', 'alvo'
    conteudo: str
    timestamp_envio: datetime
    timestamp_leitura: Optional[datetime]
    sentimento: Optional[str]  # 'positivo', 'neutro', 'negativo'

@dataclass
class Conversa:
    """Aggregate Root: Conversa."""

    # Identidade
    id: UUID

    # Relacionamentos
    perfil_id: UUID
    usuario_id: UUID
    mensagem_inicial_id: UUID
    estrategia_persuasao_id: UUID

    # Status
    status: str  # 'iniciada', 'aguardando_resposta', 'respondida', etc.

    # Timestamps
    timestamp_inicio: datetime
    timestamp_primeira_resposta: Optional[datetime]
    timestamp_handoff: Optional[datetime]
    timestamp_finalizacao: Optional[datetime]

    # Métricas
    tempo_resposta_segundos: Optional[int]
    numero_mensagens_bot: int = 0
    numero_mensagens_humano: int = 0
    numero_mensagens_alvo: int = 0

    # Resultado
    resultado: Optional[str]  # 'sucesso', 'falha', 'timeout', 'bloqueio'
    contato_obtido: Optional[str]  # WhatsApp, telefone, etc.
    notas: Optional[str]

    # Aggregate: lista de mensagens
    mensagens: List[Mensagem] = field(default_factory=list)

    def adicionar_mensagem(
        self,
```

```
        remetente: str,
        conteudo: str,
        sentimento: Optional[str] = None
    ) -> Mensagem:
        """Adiciona uma mensagem à conversa."""
        mensagem = Mensagem(
            id=uuid4(),
            remetente=remetente,
            conteudo=conteudo,
            timestamp_envio=datetime.utcnow(),
            timestamp_leitura=None,
            sentimento=sentimento
        )

        self.mensagens.append(mensagem)

        # Atualizar contadores
        if remetente == 'agente_escriba':
            self.numero_mensagens_bot += 1
        elif remetente == 'usuario_humano':
            self.numero_mensagens_humano += 1
        elif remetente == 'alvo':
            self.numero_mensagens_alvo += 1

            # Se é a primeira resposta do alvo
            if self.timestamp_primeira_resposta is None:
                self.timestamp_primeira_resposta = datetime.utcnow()
                self.tempo_resposta_segundos = int(
                    (self.timestamp_primeira_resposta -
self.timestamp_inicio).total_seconds()
                )
                self.status = 'respondida'

        return mensagem

    def acionar_handoff(self):
        """Aciona o handoff para controle humano."""
        if self.status != 'respondida':
            raise ValueError("Handoff só pode ser acionado após resposta")

        self.status = 'handoff_pendente'
        self.timestamp_handoff = datetime.utcnow()

    def confirmar_handoff(self):
        """Confirma que o handoff foi realizado."""
        if self.status != 'handoff_pendente':
            raise ValueError("Handoff não foi acionado")
```

```
        self.status = 'handoff_realizado'

    def finalizar(self, resultado: str, contato_obtido: Optional[str] =
None):
        """Finaliza a conversa com um resultado."""
        self.status = 'finalizada'
        self.resultado = resultado
        self.contato_obtido = contato_obtido
        self.timestamp_finalizacao = datetime.utcnow()
```

Repositories

Repository: Abstração que encapsula a lógica de acesso a dados, fornecendo uma interface de coleção para aggregates.

Python

```python
# dominio/repositories/perfil_repository.py
from abc import ABC, abstractmethod
from typing import List, Optional
from uuid import UUID
from dominio.entidades.perfil import Perfil

class PerfilRepository(ABC):
    """Interface do repositório de perfis."""

    @abstractmethod
    def criar(self, perfil: Perfil) -> Perfil:
        """Cria um novo perfil."""
        pass

    @abstractmethod
    def buscar_por_id(self, id: UUID) -> Optional[Perfil]:
        """Busca um perfil por ID."""
        pass

    @abstractmethod
    def buscar_por_url(self, url: str) -> Optional[Perfil]:
        """Busca um perfil por URL."""
        pass

    @abstractmethod
    def listar_por_status(
        self,
        status: str,
        limite: int = 50
    ) -> List[Perfil]:
```

```python
        """Lista perfis por status."""
        pass

    @abstractmethod
    def listar_qualificados(
        self,
        usuario_id: UUID,
        pontuacao_minima: float = 7.5,
        limite: int = 50
    ) -> List[Perfil]:
        """Lista perfis qualificados."""
        pass

    @abstractmethod
    def atualizar(self, perfil: Perfil) -> Perfil:
        """Atualiza um perfil existente."""
        pass

    @abstractmethod
    def deletar(self, id: UUID) -> bool:
        """Deleta um perfil."""
        pass

# Implementação PostgreSQL
# infraestrutura/repositorios/perfil_repository_postgresql.py
import psycopg2
from psycopg2.extras import RealDictCursor
from typing import List, Optional
from uuid import UUID
from dominio.repositories.perfil_repository import PerfilRepository
from dominio.entidades.perfil import Perfil

class PerfilRepositoryPostgreSQL(PerfilRepository):
    """Implementação do repositório de perfis usando PostgreSQL."""

    def __init__(self, connection_string: str):
        self.conn = psycopg2.connect(connection_string)

    def criar(self, perfil: Perfil) -> Perfil:
        """Cria um novo perfil."""
        with self.conn.cursor() as cur:
            cur.execute("""
                INSERT INTO perfis (
                    id, url, plataforma, username, nome_completo, biografia,
                    localizacao, foto_perfil_url, seguidores, seguindo,
                    total_postagens, status_processamento, timestamp_coleta,
                    hotspot_origem_id, usuario_id
                ) VALUES (
```

```
                    %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
%s
                )
            """, (
                perfil.id, perfil.url, perfil.plataforma, perfil.username,
                perfil.nome_completo, perfil.biografia, perfil.localizacao,
                perfil.foto_perfil_url, perfil.seguidores, perfil.seguindo,
                perfil.total_postagens, perfil.status_processamento,
                perfil.timestamp_coleta, perfil.hotspot_origem_id,
                perfil.usuario_id
            ))
            self.conn.commit()

        return perfil

    def buscar_por_id(self, id: UUID) -> Optional[Perfil]:
        """Busca um perfil por ID."""
        with self.conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute("SELECT * FROM perfis WHERE id = %s", (id,))
            row = cur.fetchone()

            if row:
                return self._row_to_perfil(row)
            return None

    def buscar_por_url(self, url: str) -> Optional[Perfil]:
        """Busca um perfil por URL."""
        with self.conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute("SELECT * FROM perfis WHERE url = %s", (url,))
            row = cur.fetchone()

            if row:
                return self._row_to_perfil(row)
            return None

    def listar_por_status(
        self,
        status: str,
        limite: int = 50
    ) -> List[Perfil]:
        """Lista perfis por status."""
        with self.conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute("""
                SELECT * FROM perfis
                WHERE status_processamento = %s
                ORDER BY timestamp_coleta DESC
                LIMIT %s
            """, (status, limite))
```

```python
        rows = cur.fetchall()
        return [self._row_to_perfil(row) for row in rows]

    def listar_qualificados(
        self,
        usuario_id: UUID,
        pontuacao_minima: float = 7.5,
        limite: int = 50
    ) -> List[Perfil]:
        """Lista perfis qualificados."""
        with self.conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute("""
                SELECT * FROM perfis
                WHERE usuario_id = %s
                  AND status_processamento = 'qualificado'
                  AND pontuacao_final >= %s
                ORDER BY pontuacao_final DESC
                LIMIT %s
            """, (usuario_id, pontuacao_minima, limite))

            rows = cur.fetchall()
            return [self._row_to_perfil(row) for row in rows]

    def atualizar(self, perfil: Perfil) -> Perfil:
        """Atualiza um perfil existente."""
        with self.conn.cursor() as cur:
            cur.execute("""
                UPDATE perfis SET
                    username = %s,
                    nome_completo = %s,
                    biografia = %s,
                    localizacao = %s,
                    foto_perfil_url = %s,
                    seguidores = %s,
                    seguindo = %s,
                    total_postagens = %s,
                    pontuacao_interessante = %s,
                    pontuacao_bonita = %s,
                    pontuacao_solteira = %s,
                    pontuacao_final = %s,
                    confianca = %s,
                    ranking = %s,
                    status_processamento = %s,
                    timestamp_analise = %s,
                    timestamp_contato = %s,
                    timestamp_resposta = %s
                WHERE id = %s
```

```
            """, (
                perfil.username, perfil.nome_completo, perfil.biografia,
                perfil.localizacao, perfil.foto_perfil_url,
perfil.seguidores,
                perfil.seguindo, perfil.total_postagens,
                perfil.pontuacao_interessante, perfil.pontuacao_bonita,
                perfil.pontuacao_solteira, perfil.pontuacao_final,
                perfil.confianca, perfil.ranking,
perfil.status_processamento,
                perfil.timestamp_analise, perfil.timestamp_contato,
                perfil.timestamp_resposta, perfil.id
            ))
            self.conn.commit()

        return perfil

    def deletar(self, id: UUID) -> bool:
        """Deleta um perfil."""
        with self.conn.cursor() as cur:
            cur.execute("DELETE FROM perfis WHERE id = %s", (id,))
            self.conn.commit()
            return cur.rowcount > 0

    def _row_to_perfil(self, row: dict) -> Perfil:
        """Converte uma row do banco em uma entidade Perfil."""
        return Perfil(
            id=row['id'],
            url=row['url'],
            plataforma=row['plataforma'],
            username=row['username'],
            nome_completo=row['nome_completo'],
            biografia=row['biografia'],
            localizacao=row['localizacao'],
            foto_perfil_url=row['foto_perfil_url'],
            seguidores=row['seguidores'],
            seguindo=row['seguindo'],
            total_postagens=row['total_postagens'],
            pontuacao_interessante=row['pontuacao_interessante'],
            pontuacao_bonita=row['pontuacao_bonita'],
            pontuacao_solteira=row['pontuacao_solteira'],
            pontuacao_final=row['pontuacao_final'],
            confianca=row['confianca'],
            ranking=row['ranking'],
            status_processamento=row['status_processamento'],
            timestamp_coleta=row['timestamp_coleta'],
            timestamp_analise=row['timestamp_analise'],
            timestamp_contato=row['timestamp_contato'],
            timestamp_resposta=row['timestamp_resposta'],
```

```
            hotspot_origem_id=row['hotspot_origem_id'],
            usuario_id=row['usuario_id']
        )
```

2.3. Design Patterns Aplicados

Além dos padrões arquiteturais de alto nível, o Projeto Quimera utiliza diversos **design patterns** clássicos (Gang of Four) e modernos para resolver problemas específicos de design.

2.3.1. Strategy Pattern

Usado no Motor de Persuasão para selecionar dinamicamente a estratégia de comunicação baseada no perfil-alvo.

Python

```python
# motor_persuasao/estrategias.py
from abc import ABC, abstractmethod
from dominio.entidades.perfil import Perfil
from dominio.value_objects.ponto_afinidade import PontoAfinidade
from typing import List

class EstrategiaPersuasao(ABC):
    """Interface para estratégias de persuasão."""

    @abstractmethod
    def construir_abordagem(
        self,
        perfil: Perfil,
        afinidades: List[PontoAfinidade]
    ) -> str:
        """Constrói a abordagem de comunicação."""
        pass

    @abstractmethod
    def identificar_gatilhos_emocionais(
        self,
        perfil: Perfil
    ) -> List[str]:
        """Identifica gatilhos emocionais relevantes."""
        pass

class EstrategiaAfeicao(EstrategiaPersuasao):
    """Estratégia baseada no princípio de Afeição (Cialdini)."""
```

```python
    def construir_abordagem(
        self,
        perfil: Perfil,
        afinidades: List[PontoAfinidade]
    ) -> str:
        """Constrói abordagem focada em pontos em comum."""
        # Selecionar a afinidade mais forte
        afinidade_principal = max(afinidades, key=lambda a: a.peso)

        abordagem = f"""
        Princípio: Afeição (Liking)
        Fundamento: Pessoas gostam de quem é similar a elas.

        Abordagem:
        - Mencionar o ponto de afinidade principal:
{afinidade_principal.descricao}
        - Enfatizar similaridades e interesses comuns
        - Usar linguagem calorosa e amigável
        - Demonstrar empatia e compreensão

        Tom: Casual, amigável, genuíno
        """

        return abordagem

    def identificar_gatilhos_emocionais(
        self,
        perfil: Perfil
    ) -> List[str]:
        """Identifica gatilhos emocionais para afeição."""
        gatilhos = []

        if perfil.biografia and 'viagem' in perfil.biografia.lower():
            gatilhos.append('curiosidade_viagens')

        if perfil.biografia and any(hobby in perfil.biografia.lower()
                                    for hobby in ['musica', 'arte',
'fotografia']):
            gatilhos.append('paixao_artistica')

        gatilhos.append('validacao_social')  # Sempre aplicável

        return gatilhos

class EstrategiaProvaSocial(EstrategiaPersuasao):
    """Estratégia baseada no princípio de Prova Social (Cialdini)."""
```

```
    def construir_abordagem(
        self,
        perfil: Perfil,
        afinidades: List[PontoAfinidade]
    ) -> str:
        """Constrói abordagem focada em conexões mútuas."""
        # Buscar afinidades de tipo 'seguidor_comum'
        seguidores_comuns = [a for a in afinidades if a.tipo ==
'seguidor_comum']

        abordagem = f"""
        Princípio: Prova Social (Social Proof)
        Fundamento: Pessoas confiam em quem é validado por sua rede social.

        Abordagem:
        - Mencionar conexões mútuas: {len(seguidores_comuns)} seguidores em
comum
        - Referenciar contexto social compartilhado
        - Demonstrar que você faz parte do mesmo círculo social

        Tom: Confiante, sociável, conectado
        """

        return abordagem

    def identificar_gatilhos_emocionais(
        self,
        perfil: Perfil
    ) -> List[str]:
        """Identifica gatilhos emocionais para prova social."""
        return [
            'pertencimento_social',
            'validacao_grupo',
            'fomo'  # Fear of Missing Out
        ]

class EstrategiaReciprocidade(EstrategiaPersuasao):
    """Estratégia baseada no princípio de Reciprocidade (Cialdini)."""

    def construir_abordagem(
        self,
        perfil: Perfil,
        afinidades: List[PontoAfinidade]
    ) -> str:
        """Constrói abordagem oferecendo valor primeiro."""
        abordagem = """
        Princípio: Reciprocidade (Reciprocity)
        Fundamento: Pessoas sentem obrigação de retribuir favores.
```

```
        Abordagem:
        - Oferecer algo de valor primeiro (informação, recomendação, elogio
genuíno)
        - Fazer um elogio específico e autêntico
        - Compartilhar algo útil relacionado aos interesses dela

        Tom: Generoso, prestativo, autêntico
        """

        return abordagem

    def identificar_gatilhos_emocionais(
        self,
        perfil: Perfil
    ) -> List[str]:
        """Identifica gatilhos emocionais para reciprocidade."""
        return [
            'gratidao',
            'obrigacao_social',
            'validacao_ego'
        ]

# Seletor de Estratégia
class SeletorEstrategia:
    """Seleciona a estratégia mais adequada para um perfil."""

    def selecionar(
        self,
        perfil: Perfil,
        afinidades: List[PontoAfinidade]
    ) -> EstrategiaPersuasao:
        """Seleciona estratégia baseada em análise do perfil."""

        # Contar tipos de afinidades
        total_afinidades = len(afinidades)
        seguidores_comuns = len([a for a in afinidades if a.tipo ==
'seguidor_comum'])
        interesses_comuns = len([a for a in afinidades if a.tipo ==
'interesse_comum'])

        # Lógica de seleção
        if seguidores_comuns >= 3:
            # Muitos seguidores em comum → Prova Social
            return EstrategiaProvaSocial()

        elif interesses_comuns >= 2 or total_afinidades >= 4:
            # Muitos interesses em comum → Afeição
```

```
            return EstrategiaAfeicao()

        else:
            # Padrão → Reciprocidade
            return EstrategiaReciprocidade()
```

2.3.2. Observer Pattern

Usado para notificar múltiplos componentes quando eventos importantes ocorrem.

Python

```python
# infraestrutura/observadores/event_observer.py
from abc import ABC, abstractmethod
from typing import List, Dict, Any

class EventObserver(ABC):
    """Interface para observadores de eventos."""

    @abstractmethod
    def atualizar(self, evento: Dict[str, Any]):
        """Método chamado quando um evento ocorre."""
        pass

class MetricsObserver(EventObserver):
    """Observador que coleta métricas de eventos."""

    def __init__(self, prometheus_registry):
        self.registry = prometheus_registry
        self.contador_eventos = Counter(
            'eventos_total',
            'Total de eventos por tipo',
            ['event_type'],
            registry=self.registry
        )

    def atualizar(self, evento: Dict[str, Any]):
        """Incrementa contador de métricas."""
        event_type = evento['event_type']
        self.contador_eventos.labels(event_type=event_type).inc()

class AuditLogObserver(EventObserver):
    """Observador que registra eventos em log de auditoria."""

    def __init__(self, logger):
        self.logger = logger
```

```python
    def atualizar(self, evento: Dict[str, Any]):
        """Registra evento em log."""
        self.logger.info(
            "Evento de auditoria",
            extra={
                "event_type": evento['event_type'],
                "event_id": evento['event_id'],
                "source": evento['source'],
                "timestamp": evento['timestamp'],
                "data": evento['data']
            }
        )

class EventSubject:
    """Subject que notifica observadores."""

    def __init__(self):
        self._observers: List[EventObserver] = []

    def anexar(self, observer: EventObserver):
        """Anexa um observador."""
        if observer not in self._observers:
            self._observers.append(observer)

    def desanexar(self, observer: EventObserver):
        """Desanexa um observador."""
        if observer in self._observers:
            self._observers.remove(observer)

    def notificar(self, evento: Dict[str, Any]):
        """Notifica todos os observadores."""
        for observer in self._observers:
            observer.atualizar(evento)

# Uso
subject = EventSubject()
subject.anexar(MetricsObserver(prometheus_registry))
subject.anexar(AuditLogObserver(logger))

# Quando um evento ocorre
evento = {
    "event_type": "PerfilAnalisado",
    "event_id": "123",
    "source": "agente_analista",
    "timestamp": "2025-10-10T16:30:00Z",
    "data": {"perfil_id": "abc", "pontuacao": 8.5}
```

```
}
subject.notificar(evento)
```

2.3.3. Factory Pattern

Usado para criar instâncias de agentes e serviços de forma flexível.

Python

```python
# infraestrutura/factories/agente_factory.py
from abc import ABC, abstractmethod
from typing import Dict, Any

class Agente(ABC):
    """Interface base para todos os agentes."""

    @abstractmethod
    def executar(self):
        """Executa a lógica principal do agente."""
        pass

class AgenteBatedor(Agente):
    def __init__(self, config: Dict[str, Any]):
        self.config = config

    def executar(self):
        print("Executando Agente Batedor...")

class AgenteAnalista(Agente):
    def __init__(self, config: Dict[str, Any]):
        self.config = config

    def executar(self):
        print("Executando Agente Analista...")

class AgenteFactory:
    """Factory para criar instâncias de agentes."""

    _agentes = {
        'batedor': AgenteBatedor,
        'analista': AgenteAnalista,
        # ... outros agentes
    }

    @classmethod
    def criar_agente(cls, tipo: str, config: Dict[str, Any]) -> Agente:
        """Cria uma instância de agente baseado no tipo."""
```

```python
        agente_class = cls._agentes.get(tipo)

        if not agente_class:
            raise ValueError(f"Tipo de agente inválido: {tipo}")

        return agente_class(config)

# Uso
config = {"proxy_url": "http://proxy.com", "user_agent": "Mozilla/5.0"}
agente = AgenteFactory.criar_agente('batedor', config)
agente.executar()
```

2.3.4. Circuit Breaker Pattern

Usado para prevenir cascata de falhas quando um serviço externo está indisponível.

Python

```python
# infraestrutura/resilience/circuit_breaker.py
from enum import Enum
from datetime import datetime, timedelta
from typing import Callable, Any
import time

class CircuitState(Enum):
    CLOSED = "closed"      # Funcionando normalmente
    OPEN = "open"          # Circuito aberto, rejeitando requisições
    HALF_OPEN = "half_open"  # Testando se o serviço voltou

class CircuitBreaker:
    """Implementação do padrão Circuit Breaker."""

    def __init__(
        self,
        failure_threshold: int = 5,
        timeout: int = 60,
        success_threshold: int = 2
    ):
        self.failure_threshold = failure_threshold
        self.timeout = timeout  # Tempo em segundos antes de tentar novamente
        self.success_threshold = success_threshold

        self.failure_count = 0
        self.success_count = 0
        self.last_failure_time = None
        self.state = CircuitState.CLOSED
```

```python
    def call(self, func: Callable, *args, **kwargs) -> Any:
        """Executa a função protegida pelo circuit breaker."""

        if self.state == CircuitState.OPEN:
            if self._should_attempt_reset():
                self.state = CircuitState.HALF_OPEN
            else:
                raise Exception("Circuit breaker está OPEN. Serviço
indisponível.")

        try:
            result = func(*args, **kwargs)
            self._on_success()
            return result

        except Exception as e:
            self._on_failure()
            raise e

    def _on_success(self):
        """Chamado quando a requisição é bem-sucedida."""
        self.failure_count = 0

        if self.state == CircuitState.HALF_OPEN:
            self.success_count += 1

            if self.success_count >= self.success_threshold:
                self.state = CircuitState.CLOSED
                self.success_count = 0

    def _on_failure(self):
        """Chamado quando a requisição falha."""
        self.failure_count += 1
        self.last_failure_time = datetime.now()

        if self.failure_count >= self.failure_threshold:
            self.state = CircuitState.OPEN

    def _should_attempt_reset(self) -> bool:
        """Verifica se deve tentar resetar o circuito."""
        if self.last_failure_time is None:
            return False

        elapsed = (datetime.now() - self.last_failure_time).total_seconds()
        return elapsed >= self.timeout

# Uso
circuit_breaker = CircuitBreaker(failure_threshold=5, timeout=60)
```

```python
def chamar_api_externa():
    # Simula chamada a API externa que pode falhar
    response = requests.get("https://api.externa.com/data")
    response.raise_for_status()
    return response.json()

try:
    resultado = circuit_breaker.call(chamar_api_externa)
    print(f"Sucesso: {resultado}")
except Exception as e:
    print(f"Falha: {e}")
```

2.4. Comunicação Entre Componentes

2.4.1. Comunicação Síncrona (HTTP/REST)

Usada quando uma resposta imediata é necessária.

Exemplo: API Backend → Orquestrador

Python

```python
# api_backend/rotas/perfis.py
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List
import httpx

router = APIRouter()

class IniciarProspeccaoRequest(BaseModel):
    hotspot_ids: List[str]

@router.post("/prospeccao/iniciar")
async def iniciar_prospeccao(request: IniciarProspeccaoRequest):
    """Inicia processo de prospecção."""

    # Chamada síncrona HTTP para o Orquestrador
    async with httpx.AsyncClient() as client:
        response = await client.post(
            "http://orquestrador:8001/workflow/iniciar",
            json={
                "workflow_type": "prospeccao",
                "hotspot_ids": request.hotspot_ids
            },
```

```
            timeout=10.0
        )

        if response.status_code != 200:
            raise HTTPException(
                status_code=500,
                detail="Falha ao iniciar prospecção"
            )

        return response.json()
```

2.4.2. Comunicação Assíncrona (Message Queues)

Usada para desacoplar componentes e permitir processamento assíncrono.

Exemplo: Agente Batedor → Agente Analista (via Redis)

Python

```
# Já mostrado anteriormente na seção de Event-Driven Architecture
```

2.4.3. Comunicação em Tempo Real (WebSocket)

Usada para notificações em tempo real para o frontend.

Python

```
# api_backend/websocket.py
from fastapi import WebSocket, WebSocketDisconnect
from typing import List
import asyncio

class ConnectionManager:
    """Gerencia conexões WebSocket."""

    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        """Aceita nova conexão."""
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        """Remove conexão."""
        self.active_connections.remove(websocket)
```

```python
    async def broadcast(self, message: dict):
        """Envia mensagem para todas as conexões."""
        for connection in self.active_connections:
            await connection.send_json(message)

manager = ConnectionManager()

@app.websocket("/ws/notificacoes")
async def websocket_endpoint(websocket: WebSocket):
    await manager.connect(websocket)

    try:
        while True:
            # Manter conexão aberta
            await websocket.receive_text()

    except WebSocketDisconnect:
        manager.disconnect(websocket)

# Quando uma resposta é detectada, notificar via WebSocket
async def notificar_resposta_detectada(perfil_id: str):
    await manager.broadcast({
        "type": "resposta_detectada",
        "perfil_id": perfil_id,
        "timestamp": datetime.utcnow().isoformat()
    })
```

2.5. Conclusão do Capítulo

Este capítulo aprofundou a arquitetura de software do Projeto Quimera, cobrindo:

- **Princípios Arquiteturais:** SRP, Loose Coupling, High Cohesion, Fail-Fast, Observability, Security by Design
- **Padrões Arquiteturais:** Microservices, Event-Driven Architecture, CQRS, Domain-Driven Design
- **Design Patterns:** Strategy, Observer, Factory, Circuit Breaker
- **Comunicação:** Síncrona (HTTP), Assíncrona (Message Queues), Tempo Real (WebSocket)

No próximo capítulo, mergulharemos na **infraestrutura e DevOps**, cobrindo containerização, orquestração, monitoramento e deploy.

Próximo Capítulo: Capítulo 03 - Infraestrutura e DevOps

Referências deste Capítulo:

- Fowler, M. (2014). *Microservices*
- Richardson, C. (2018). *Microservices Patterns*
- Vernon, V. (2013). *Implementing Domain-Driven Design*
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*
- Nygard, M. (2018). *Release It!: Design and Deploy Production-Ready Software*